Alterative ISAs: Legacy Upgrades

James Brakefield

My computer "**zoo**"[1] can be considered as improvements on existing architectures. Several of them are based closely on past computers with the intent to overcome their limitations. The primary limitation is insufficient address space. This is a natural result of progress in memory chip density. Other limitations are limited data types, limited instruction operations, limited data sizes, limited register file size and obsolescent addressing modes.

First, some terminology: Instead of RISC, I use **ROC**: Register Oriented Computer. That is ALU operations take place between registers in the register file with a separate set of load and store instructions. Instead of CISC, I use **MAOC**: Multiple Accumulator Oriented Computer. Operations take place within the register file or between the register file and memory. For stack machines I use the term **OOSM**: One Operand Stack Machine. This ISA crams both an operation code and a memory or stack reference into a single instruction. Usually the standard dual stack operations are included as single byte op-codes. The standard accumulator architecture is simply **Accum**. These names are chosen to be dramatic and more appropriate than standard nomenclature.

My contribution of original ISAs includes a new rendition of RISC: **TROC** where the **T** stands for Tagged. This is done to reduce the number of op-codes as TROC supports four data types and four data sizes. The tag provides two bits of type and two or more bits of additional exponent and mantissa for floating-point values[2]. The legacy ISAs being targeted by ROC and TROC are MIPS, Power PC and RISC-V. The tags are present in the register file and not in main memory.

The other original ISA targets stack machines. In order to reduce the number of instructions that need to be decoded, as much work as possible is squeezed into a single 16-bit instruction. An op-code, stack pop/push, a near stack reference and a memory addressing mode. Again there is a tagged version **TOOSM** with tags supported in the stacks but not in main memory.

The problems of some legacy computers:

DEC PDP-11: insufficient address space, insufficient number of registers and insufficient operation codes. Addressed by increasing register size, having an escape to larger instructions, and provision for several sizes of immediate values (encompassing small constants, relative displacements, memory offset values and large constants). The register file is increased to twelve registers and PC by limiting address modes on general registers to five: register, register indirect, register auto increment, register auto decrement and register indirect with offset. There are four codes remaining and are used for PC based

[1] The term "computer zoo" originates AFAIK from back cover of: *Computer Architecture, Concepts and Evolution* by Gerrit Blaauw and Frederick Brooks Jr. 1997; 1213 pages on acid free paper.

[2] Operations between register values of different types could provide type conversion. As this adds considerable complexity and variable instruction timings it is not required. E.g. the instruction traps to an exception handler.

addressing: Immediate, relative, absolute and the stack pointer. The double indirect modes can be accomplished via an additional load instruction.

TI MPS430: insufficient address space, a missing address mode and insufficient operation codes. Here the register file is reduced from sixteen to fourteen allowing a fifth address mode of register auto pre-decrement indirect.

Both alt11 and alt430 take one instruction and repurpose it for signifying a longer instruction. For alt430 it is the BCD instruction and on the alt11 it is the floating-point instruction. This allows support for additional data sizes and additional data types. The original ISAs are maintained as much as possible with the exception that of the two data sizes in the original ISAs, the 8-bit byte and the 16-bit word are changed to 8-bit byte and full register size.

The DEC VAX-780 is criticized for too much generality, e.g. instructions can be long and invoked several virtual memory references. My solution is to eliminate the double indirect address modes and make the 24-bit instruction the dominate case. Only one operand or the result can be a memory reference. Thus an eight-bit op-code, three four bit register fields and a four-bit addressing mode. The addressing mode specifies application to either one source operand or to the result. Eight addressing modes are possible. A separate prefix byte is used to specify an index register.

The x86-64 suffers from an elaborate and very dated ISA. Again I use a 24-bit instruction with provision for appended immediate values and provision for a 32-bit instruction to handle elaborate instructions.

In summary, my ALT designs tend to cluster around a 16 or 24-bit primary instruction size with byte addressing and alignment; with support for four data types and four data sizes. More complicated instructions use a 32-bit or larger instruction.

Alt11, Alt430 and Alt780 remain very close to the original's flavor with the majority of the ISA carried forward. There is limited compatibility between the new ISAs and the old except by having emulation enable in the status register. The Alt780 and Alt86 retain most of the original op-codes and most of the addressing modes. Gone are lengthy instructions, most prefix bytes, obsolete instructions and over the top addressing modes[3].

There is also a way to revamp the addressing modes that reduces the register file address updates. Instead of auto increment/decrement one introduces an index mode with an implied index register. The five modes are: register, register plus small offset indirect, register plus large offset indirect, register plus scaled implied index plus small offset indirect and register plus scaled implied index plus large offset indirect.

Major goals in all of this is computer architecture that has a minimum number of instruction formats, has consistent placement of register fields, consistent means to support multiple length immediate

[3] In the pursuit of higher performance the auto-increment and auto-decrement address modes cause more register file updates than indexed and memory offset modes. Multiple memory references per instruction increase virtual address translations and conflict with placing memory references earlier in the instruction stream.

values, and which supports high performance by means of instruction alignment to half or full word boundaries, including immediate value sizes.